SASKEN

Real-Time Multimedia System Analysis: A Breeze in Linux

Whitepaper

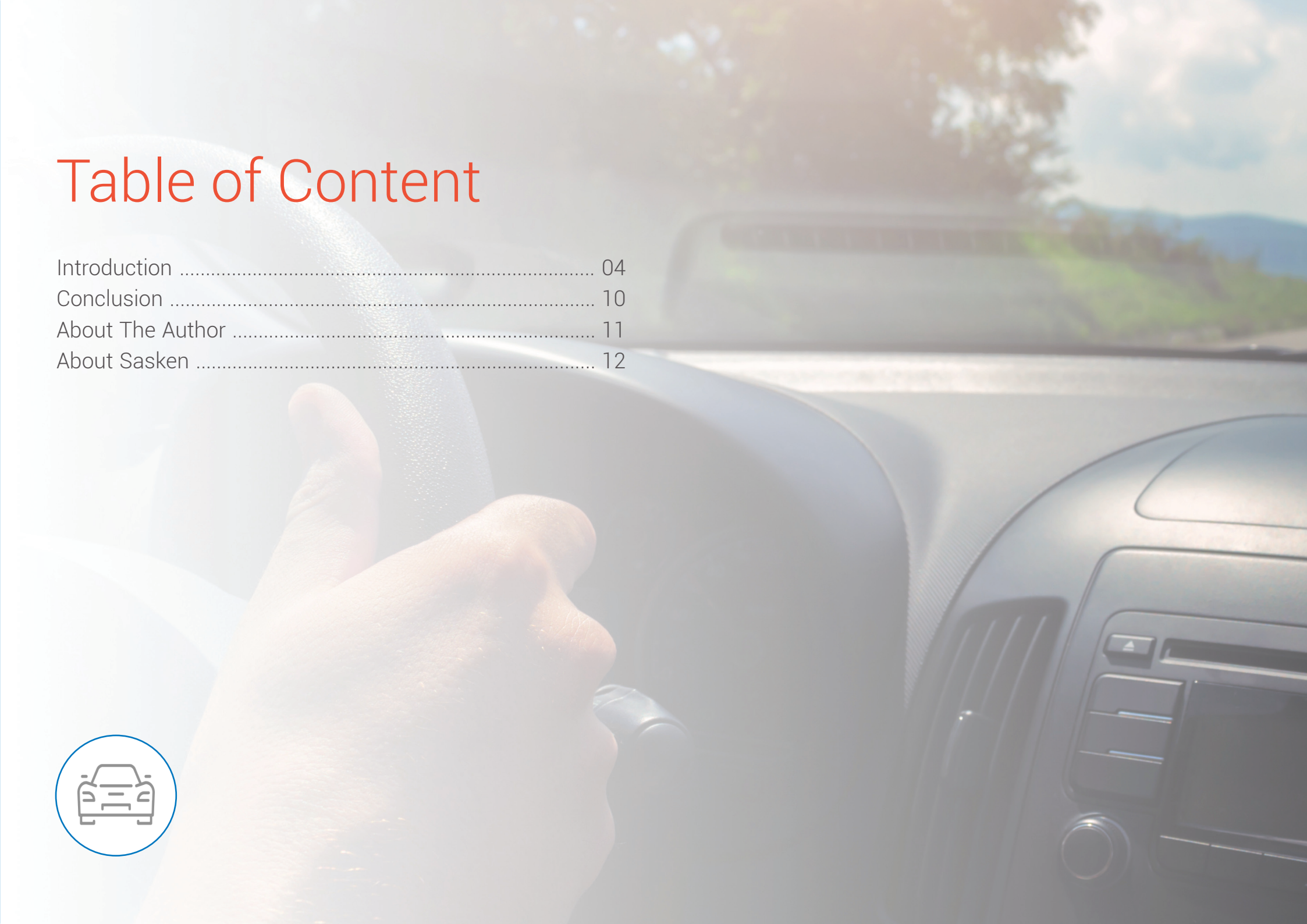
Abstract

Modern infotainment processors have sophisticated hardware accelerators for high-resolution video processing, but still there are some use-cases that stretch them to the limit, requiring near real-time processing in realizing them (e.g. BD). On general purpose operating systems (GPOS), meeting these requirements can be a challenge, resulting in video artefacts like jerkiness, freeze, audio-video sync loss, and distortion. This article will take you through a brief overview of typical media processing solution, issues faced in achieving real-time audio video rendering, and tools in Linux to analyze and fix such issues while taking the BD video/graphics data-path as an example.

Keywords - Kernel Tracers, trace-cmd, Blu-ray, de-interlace, blending

Author:
Chetan Sethi,
System Software Architect,
In-Vehicle Infotainment

Table of Content



Introduction

Typical media processing solutions like BD, STB, disc players need to process video and graphics data and render it. Different post processing operations like de-interlacing, alpha blending, color conversion, scaling, etc. are accomplished using variety of hardware accelerators to prepare one composite signal to be rendered.

Typical video/graphics sub-system post the decoding stage consists of the following:

- Post processing of video, image
- Blending videos and graphics
- Rendering video, image, graphics

BD composite signal consists of primary video, secondary video, and graphics. For interlaced video content, de-interlacing operation is performed and then primary video, secondary video, and BD graphics signals are blended. This composite BD signal is then finally blended with user graphics before rendering onto the LCD display.

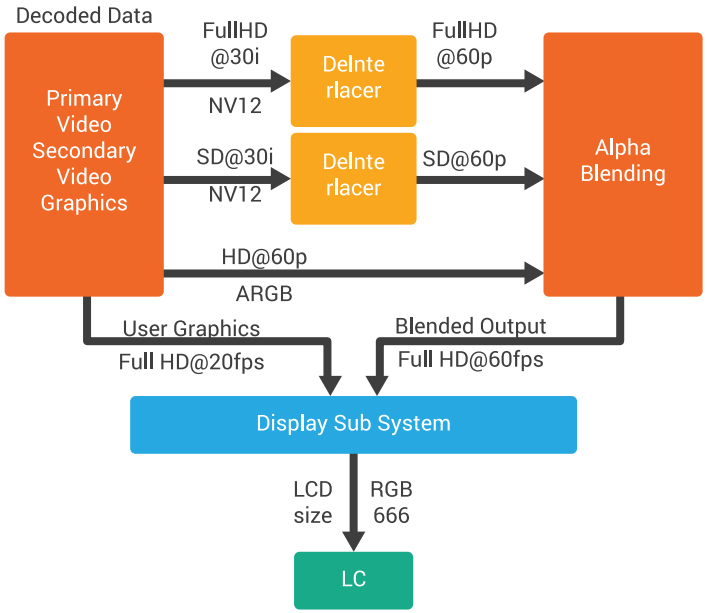


Figure 1: Typical BD Data Path

In a general software architecture design for video/graphics composite signal rendering use cases, parallel execution of independent hardware accelerators like de-interlacer, Graphics Processing Unit (2S/3D Acceleration)/Blender and display sub-system are highly desirable to achieve optimal performance and throughput. The key idea is to reduce the interdependency, define independent features/task so that the execution is done in true parallel way to feed data at real-time intervals.

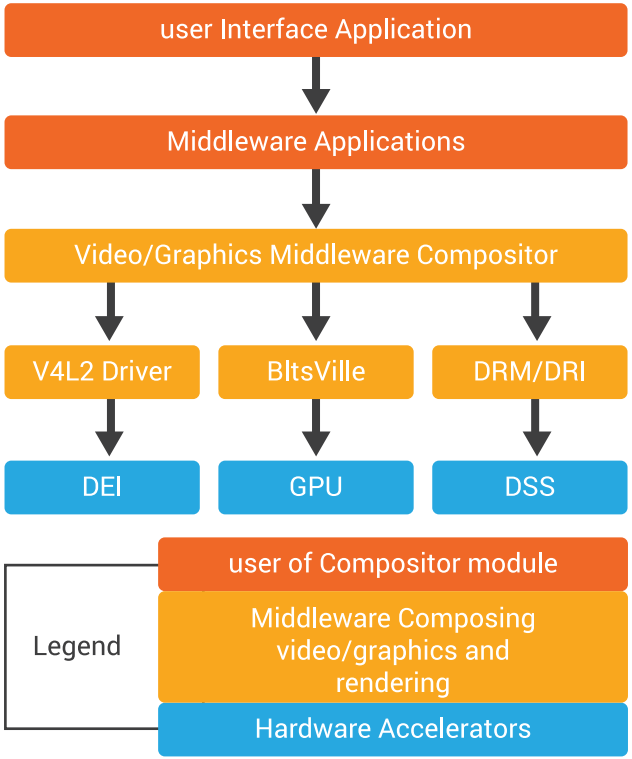


Figure 2: Typical BD Software stack

At the software level, a pipeline based threading model can be used for effective usage of these accelerators by defining one execution context (thread) per hardware accelerator. These threads are responsible picking up buffer for processing, executing post processing operation using the hardware blocks and outputting the resultant buffer to next hardware module. These execution contexts can have input and output queues defined so that, output of each stage can be fed into input of another stage.

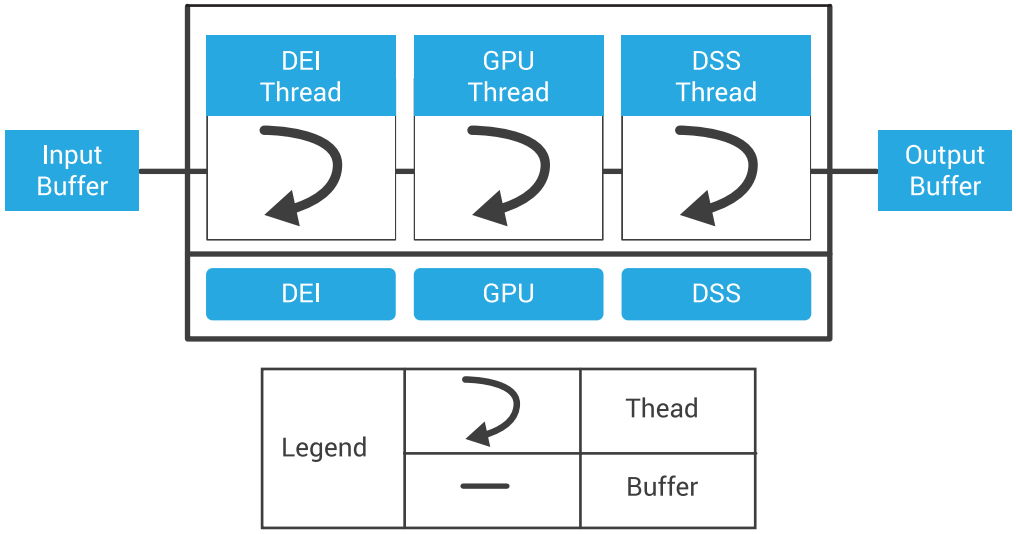


Figure 3: Pipeline based threading model

The software stack for video/graphics sub-system needs to ensure that it is capable of rendering the frames at LCD refresh rate. Issues like jerky video, audio-video sync loss, distortion, video freeze, black screen can be noticed while achieving real-time rendering. Multiple reasons like frame drops, reception of delayed frames at rendering level, multiple user-kernel transactions, scheduling issues, locking contentions, clock related issues, high interrupt/DMA loads, etc. can result in these.

Finding the root cause of above mentioned artefacts, invariably requires an insight into what the system as a whole is doing at the instant of the artefact. Logs are of limited use when the analysis has to span multiple independently-developed components, system behavior, and long duration execution.

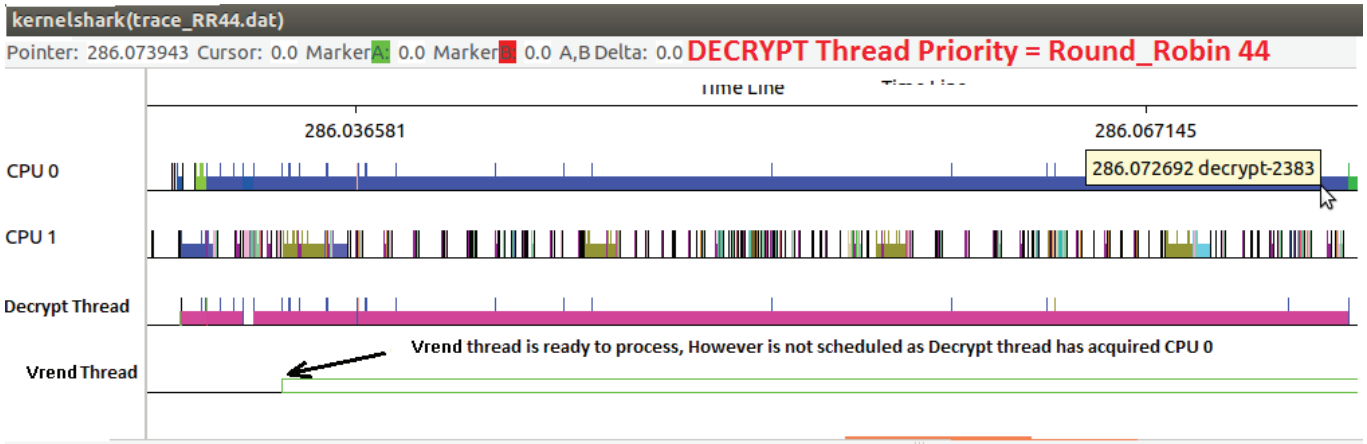
Kernel tracing in Linux provides a light-weight, time-accurate capability to inspect system events occurring between specific points in the application execution. This information is invaluable in ascertaining the system's impact on the performance of a specific application and identifying the bottlenecks. In Linux these can be achieved via trace-cmd, tool that initiates the trace operation and generates the log and KernelShark, a GUI front-end tool for visual inspection of the log.

To ensure, the rendering is done flawless and in real-time, it's essential that each block in video/graphics software pipeline does its unit task in well-defined real-time boundary. To start with ensuring this, Kernel traces can be used for accurate time measurement of different hardware IPs.

```
  SAS_OMX_VREND-2190  [000]   180.499237: print:                 c00ac008 VPE_ENTER[0][1][2]_numofbohandles 0xa90a43b0 0x5ec400 (nil) (nil) 1
  SAS_OMX_VREND-2190  [000]   180.499268: print:                 c00ac008 vpe   : Locked   : 0x6546c8
  SAS_OMX_VREND-2190  [000]   180.499390: print:                 c00ac008 vpe   : Locked   : 0x654788
  SAS_OMX_VREND-2190  [000]   180.499481: print:  Deinterlacer   c00ac008 app   : Locked   : 0x5ec400
                                                  Entry-Exit
   SAS_OMX_VDEC-3669  [001]   180.499939: print:                 c00ac008 VDEC_CTNStat@T_handle 0xb230cde8 : MsgP_IO 58 10 Put_IO 1376 670 Get_IO 1318 66
   SAS_OMX_VDEC-3669  [001]   180.500092: print:                 c00ac008 0x802116c8 0xb230cde8 208335411
vdec_1.output_0-3671  [000]   180.500427: print:                 c00ac008 V_ETB_Port_pBH-pB_L_F_PTS_CMT 1 0x5ebbc0 0x5ebc60 3637248 0x50000 26679544 2650
      VIDEOREND-2202  [000]   180.501160: print:                 c00ac008 gc320 : Locked   : 0x527228
      VIDEOREND-2202  [000]   180.501190: print:                 c00ac008 GC320_ENTER 3
  SAS_OMX_VREND-2190  [000]   180.502594: print:                 c00ac008 VPE_EXIT[0][1][2]_numofbohandles 0xa90a43b0 0x5ec400 (nil) (nil) 1
  SAS_OMX_VREND-2190  [000]   180.502594: print:                 c00ac008 VGM_REC_BUFFER_ENTER_i_vgmhandle_nbufzise 0 0x5f7150 0
  SAS_OMX_VREND-2190  [000]   180.502625: print:  GPU Entry-Exit c00ac008 VGM_REC_BUFFER_EXIT_i_vgmhandle_bo_handle_nbufzise 0 0x5f7150 0x5eaf28 3637248
   SAS_OMX_VDEC-3669  [000]   180.502838: print:                 c00ac008 VIDDEC3_Process_EXIT 0x802116c8 0xb230cde8 208335411
  SAS_OMX_VDEC-3646   [000]   180.504547: print:                 c00ac008 VDEC_CTNStat@T_handle 0x90900268 : MsgP_IO 71 16 Put_IO 1376 689 Get_IO 1305 67
      VIDEOREND-2202  [000]   180.508240: print:                 c00ac008 GC320_EXIT 3
      VIDEOREND-2202  [000]   180.508301: print:                 c00ac008 gc320 : Locked   : 0x526fb8
```

Figure 4: Time Measurement (Command: #trace-cmd record –e ftrace)

Kernel traces can be used for figuring out scheduling issues like higher priority thread of one particular process not allowing video rendering thread belonging to other process to schedule, leading to video breaks/jerky video issues. Such issues can be resolved by appropriately tuning the thread priorities of both the threads.

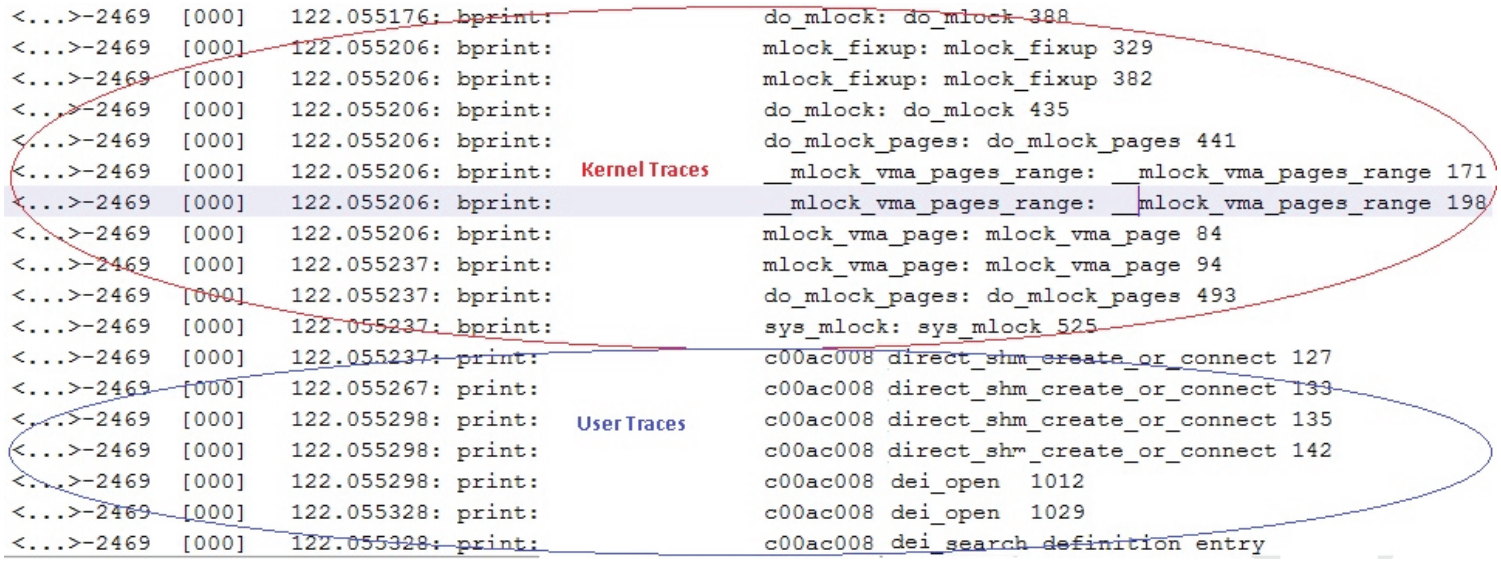


Page 1 Search: Column: # contains graph follows

#	CPU	Time Stamp	Task	PID	Latency	Event	Info
0	0	278.570251	trace-cmd	4682	d.h5	sched_wakeup	4682:?:? + 4682:120:? trace-cmd [000] Success
1	0	278.570281	trace-cmd	4682	d.h5	sched_stat_runtime	comm=trace-cmd pid=4682 runtime=427246 [ns] vrunti
2	0	278.570281	trace-cmd	4682	d.h5	sched_stat_sleep	comm=kworker/0:0 pid=4 delay=33660889 [ns]
3	0	278.570281	trace-cmd	4682	d.h5	sched_wakeup	4682:?:? + 4:120:? kworker/0:0 [000] Success
4	0	278.570281	trace-cmd	4682	dN.3	sched_stat_wait	comm=kworker/0:0 pid=4 delay=0 [ns]
5	0	278.570281	trace-cmd	4682	d..3	sched_switch	4682:120:R ==> 4:120: kworker/0:0
6	0	278.570312	kworker/0:0	4	d..3	sched_stat_runtime	comm=kworker/0:0 pid=4 runtime=30517 [ns] vruntime
7	0	278.570312	kworker/0:0	4	d..3	sched_stat_wait	comm=trace-cmd pid=4682 delay=30517 [ns]
8	0	278.570312	kworker/0:0	4	d..3	sched_switch	4:120:S ==> 4682:120: trace-cmd
9	0	278.570312	trace-cmd	4682	d.h5	sched_stat_sleep	comm=kworker/0:0 pid=4 delay=0 [ns]
10	0	278.570343	trace-cmd	4682	d.h5	sched_wakeup	4682:?:? + 4:120:? kworker/0:0 [000] Success
11	0	278.570343	trace-cmd	4682	dN.3	sched_stat_wait	comm=kworker/0:0 pid=4 delay=0 [ns]
12	0	278.570343	trace-cmd	4682	d..3	sched_switch	4682:120:R ==> 4:120: kworker/0:0
13	0	278.570343	kworker/0:0	4	d..3	sched_stat_runtime	comm=kworker/0:0 pid=4 runtime=30518 [ns] vruntime
14	0	278.570343	kworker/0:0	4	d..3	sched_stat_wait	comm=trace-cmd pid=4682 delay=30518 [ns]
15	0	278.570343	kworker/0:0	4	d..3	sched_switch	4:120:S ==> 4682:120: trace-cmd

Figure 5: Scheduling Issues (Command: #trace-cmd record –e sched* -o sched.dat#kernelshark sched.dat)

Kernel tracers have distinctive capability of common logging between user space and kernel space which can be extremely useful in figuring out issues like video freeze due to locking contentions at application layer.




```
<...>-2469  [000]  122.055176: bprint:   do_mlock: do_mlock 388
<...>-2469  [000]  122.055206: bprint:   mlock_fixup: mlock_fixup 329
<...>-2469  [000]  122.055206: bprint:   mlock_fixup: mlock_fixup 382
<...>-2469  [000]  122.055206: bprint:   do_mlock: do_mlock 435
<...>-2469  [000]  122.055206: bprint:   do_mlock_pages: do_mlock_pages 441
<...>-2469  [000]  122.055206: bprint:   __mlock_vma_pages_range: __mlock_vma_pages_range 171
<...>-2469  [000]  122.055206: bprint:   __mlock_vma_pages_range: __mlock_vma_pages_range 198
<...>-2469  [000]  122.055206: bprint:   mlock_vma_page: mlock_vma_page 84
<...>-2469  [000]  122.055237: bprint:   mlock_vma_page: mlock_vma_page 94
<...>-2469  [000]  122.055237: bprint:   do_mlock_pages: do_mlock_pages 493
<...>-2469  [000]  122.055237: bprint:   sys_mlock: sys_mlock 525
<...>-2469  [000]  122.055237: print:    c00ac008 direct_shm_create_or_connect 127
<...>-2469  [000]  122.055267: print:    c00ac008 direct_shm_create_or_connect 133
<...>-2469  [000]  122.055298: print:    c00ac008 direct_shm_create_or_connect 135
<...>-2469  [000]  122.055298: print:    c00ac008 direct_shm_create_or_connect 142
<...>-2469  [000]  122.055298: print:    c00ac008 dei_open  1012
<...>-2469  [000]  122.055328: print:    c00ac008 dei_open  1029
<...>-2469  [000]  122.055328: print:    c00ac008 dei_search definition entry
```

Figure 6: User/Kernel Logging (Command: #trace-cmd record –e ftrace)

Kernel tracers provide interrupt statistics that can be captured using trace-cmd. Also trace-cmd extends interface for easy adaptation of data in excel format for further analysis. As can be seen in below example, one interrupt occurs per milliseconds indicating a very high rate of unexpected interruption adding to un-necessary load on the system. This particular instance of high rate of interrupt was traced to a power management issue in one of the video post processing hardware IPs.

Time Stamp	Function name	IRQ No	Interrupt Name
1065.393695	irq_handler_entry	irq=187	name=48468000.dei
1065.394736	irq_handler_entry	irq=187	name=48468000.dei
1065.395776	irq_handler_entry	irq=187	name=48468000.dei
1065.396817	irq_handler_entry	irq=187	name=48468000.dei
1065.397861	irq_handler_entry	irq=187	name=48468000.dei
1065.398904	irq_handler_entry	irq=187	name=48468000.dei
1065.399954	irq_handler_entry	irq=187	name=48468000.dei
1065.400998	irq_handler_entry	irq=187	name=48468000.dei
1065.402035	irq_handler_entry	irq=187	name=48468000.dei
1065.403076	irq_handler_entry	irq=187	name=48468000.dei
1065.404123	irq_handler_entry	irq=187	name=48468000.dei

Figure 7 Interrupt statistics (#trace-cmd record –e irq)

Kernel tracers assist in analyzing system calls. As can be seen in below example, one particular system call (NR 91- sysmunmap) is taking around 235 milliseconds of time for returning. Such method is extremely useful in analyzing system specific issues like tuning of DMA channel priorities.

TID	TimeStamp	System Call	Details
1944	1439.713837	sys_enter	NR 91 (95b03000, 1c2000, 0, 0)
1944	1439.713867	irq_handler_entry	irq=30 name=arch_timer
1944	1439.713867	hrtimer_cancel	hrtimer=0xca6b3f28
1944	1439.713867	hrtimer_expire_entry	hrtimer=0xca6b3f28
1944	1439.713867	sched_stat_sleep	comm=trace-cmd pid=17003
1944	1439.79306	sched_wakeup	1944?? + 1946100?
1944	1439.79306	hrtimer_expire_exit	hrtimer=0xe9fadac8
1944	1439.79306	irq_handler_exit	irq=30 ret=handled
1944	1439.79892	softirq_entry	vec=1 [action=TIMER]
1944	1439.79895	softirq_exit	vec=1 [action=TIMER]
1944	1439.806702	irq_handler_entry	irq=30 name=arch_timer
1944	1439.806732	hrtimer_expire_entry	hrtimer=0xc1211a98
1944	1439.806732	softirq_raise	vec=1 [action=TIMER]
1944	1439.806732	rcu_utilization	c078c644
1944	1439.949371	kfree	(drm_gem_vm_close+0x28)
1944	1439.949371	kmem_cache_free	(remove_vma+0x48)
1944	1439.949371	sys_exit	NR 91 = 0
1944	1439.713837	sys_enter	NR 91 (95b03000, 1c2000, 0, 0)

Figure 8 System Call statistics (#trace-cmd record –e all)

Conclusion

Kernel-shark and trace-cmd are ideal tools for analyzing performance related issues/latencies and they have quite a few advantages for performance measurement over traditional methods. Apart from performance measurement quite a few advance debugging techniques are exposed by them.

The key advantages of the trace-cmd and KernelShark tools are:

- It can be used for debugging or analyzing latencies and performance issues.
- It has minimal impact on the CPU load and is very accurate.
- It has ability to trace syscall entry and exit, and signal delivery, to a process (which can also used for debugging a process)
- Excellent user level representation tools like kernel shark.
- Dynamic control for recording/stopping traces.
- Complete thread level scheduling statistics for the entire system can be generated.
- Provide interface for plotting graphs in excel sheet.
- It gives detailed information on multiple system parameters like interrupts, kernel events etc.

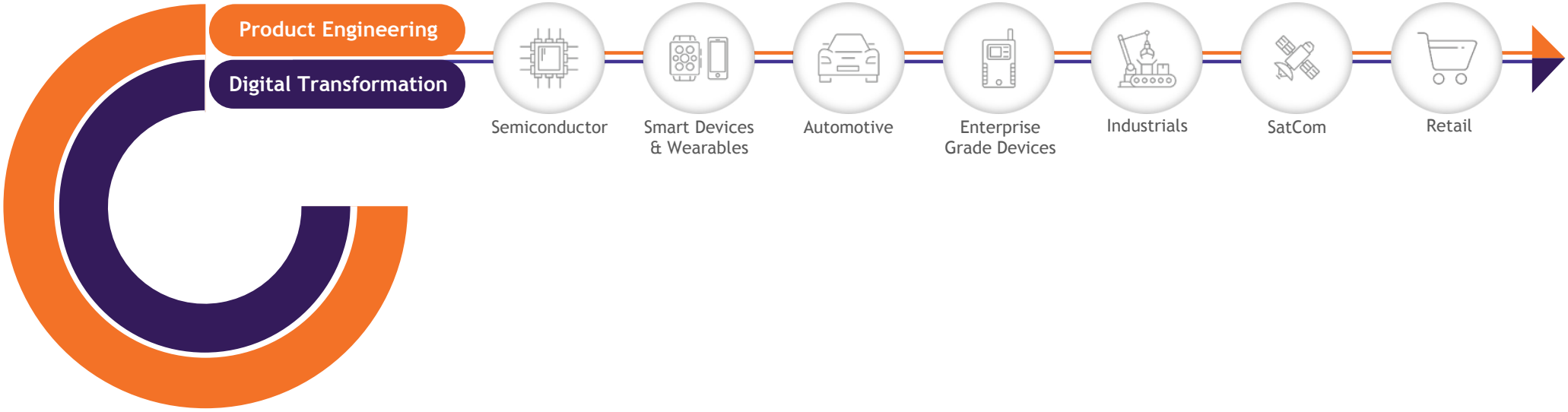
Gives unified interface for logging across kernel and user space with single log file generation.

About The Author

Chetan Sethi has rich experience in Automotive In-Vehicle Infotainment product development, Linux, Multimedia, and the Consumer Electronics space. He has technical contribution in multiple areas spanning from performance analysis of complex systems, base porting in Linux, audio DSP algorithm development to name a few. Also he has immense exposure of working with Japanese
Tier-1s for entire product development cycle without compromising upon stringent quality expectations and strict deadlines. At Sasken, has the responsibility of delivering automotive In-Vehicle Infotainment sub-system for future models to a Japanese Tier-1.

About Sasken

Sasken is a specialist in Product Engineering and Digital Transformation providing concept-to-market, chip-to-cognition R&D services to global leaders in Semiconductor, Automotive, Industrials, Smart Devices & Wearables, Enterprise Grade Devices, Satcom, and Retail industries. With over 27 years in Product Engineering and Digital Transformation and 70 patents, Sasken has transformed the businesses of over a 100 Fortune 500 companies, powering over a billion devices through its services and IP.



SASKEN

Real-Time Multimedia System Analysis: A Breeze in Linux

marketing@sasken.com | www.sasken.com

USA | UK | FINLAND | GERMANY | JAPAN | INDIA

June 2017